

MO4 & MO5:

Wiki Systems Manual

Group Members:

Abdul Baari Davids - ST10267411

Ethan Smyth - ST10255309

Ethan Donaldson - ST10318621

Jacques du Plessis - ST10329686

Joshua Wood (PM) - ST102961671

Table of Contents

1. System Overview.....	2
1.1 Purpose.....	2
1.3 Key Components.....	2
1.4 Content Structure.....	3
1.5 System Requirements.....	4
2. Deployment & Hosting.....	5
2.1 Cloudflare Pages Setup.....	5
Initial Configuration (One-Time).....	5
Custom Domain Configuration.....	6
2.2 GitHub Actions Workflow.....	7
2.3 Deployment Workflow.....	9
Production Deployment.....	9
Preview Deployments.....	10
3. Configuration Management.....	11
3.1 Hugo Configuration.....	11
3.2 Docsy Theme Configuration.....	12
3.3 Content Front Matter.....	13
3.4 Search Configuration.....	14
4. Content Management.....	15
4.1 Writing Guidelines.....	15
4.2 Adding New Content.....	16
Adding a New FAQ.....	16
Adding a Blog Post.....	17
4.3 Adding Images.....	17
4.4 Content Organization.....	18
4.5 Content Review Process.....	19
5. Build & Release Process.....	20
5.1 Local Development.....	20
5.2 Production Build.....	21
6. Monitoring & Performance.....	22
6.1 Performance Metrics.....	22
7. Search Functionality.....	23
7.1 Search Index Generation.....	23
7.2 Client-Side Search (Lunr.js).....	24
7.3 Search Optimization.....	24
7.4 Search Analytics.....	25

1. System Overview

1.1 Purpose

The ZATech Community Wiki serves as the centralized knowledge base for the ZATech community, providing:

- **FAQs:** Frequently asked questions and answers
- **Channel Guides:** Information about Slack channels
- **Community Guidelines:** Rules, etiquette, best practices
- **Technical Guides:** Tutorials, how-tos, technical documentation
- **Blog Posts:** Longer-form content, announcements, community stories

1.2 Key Components

Component	Purpose	Technology
Static Site Generator	Convert Markdown → HTML	Hugo Extended 0.150+
Theme	UI/UX framework	Docsy (Hugo theme)
Content Format	Documentation source	Markdown + YAML front matter
Search	Client-side search	Lunr.js (JavaScript)
CDN	Global content delivery	Cloudflare Pages
CI/CD	Automated builds	GitHub Actions
Version Control	Content management	Git + GitHub

1.3 Content Structure

```
content/
├── faqs/                                # Frequently Asked Questions
│   ├── _index.md
│   ├── how-to-invite.md
│   └── channel-list.md
├── channels/                            # Channel-specific pages
│   ├── _index.md
│   ├── general.md
│   └── tech-discussions.md
├── pages/                              # General pages
│   ├── _index.md
│   ├── guidelines.md
│   └── commercial-channels.md
└── blog/                               # Blog posts
    ├── _index.md
    ├── 2025-community-update.md
    └── welcome-guide.md
```

1.4 System Requirements

Build Environment:

- **Hugo Extended:** $\geq 0.150.0$ (with Sass support)
- **Go:** ≥ 1.21 (for Hugo Modules)
- **Node.js:** ≥ 18 (for PostCSS, used by Docsy theme)
- **Git:** Latest version

Local Development:

- macOS, Linux, or Windows (with WSL recommended)
 - 2 GB RAM minimum
 - 1 GB free disk space
-

2. Deployment & Hosting

2.1 Cloudflare Pages Setup

Initial Configuration (One-Time)

1. **Create Cloudflare Account:**

- Visit <https://dash.cloudflare.com/sign-up>
- Verify email

2. **Connect GitHub Repository:**

- Cloudflare Dashboard → Pages → Create a project
- Connect to Git → Authorize Cloudflare
- Select repository: `zatech-wiki`

3. **Build Settings:**

Setting	Value
Project name	<code>zatech-wiki</code>
Production branch	<code>main</code>
Build command	<code>npm install && hugo --minify --gc</code>
Build output directory	<code>public</code>
Root directory	<code>/</code>

4. **Environment Variables:**

Variable	Value	Purpose
<code>HUGO_VERSION</code>	<code>0.150.0</code>	Hugo version to use
<code>NODE_VERSION</code>	<code>18</code>	Node.js version for PostCSS

Variable	Value	Purpose
GO_VERSION	1.21	Go version for modules

5. Deploy:

- Click **Save and Deploy**
- First build starts automatically (~60-90 seconds)

Custom Domain Configuration

1. Add Custom Domain:

- Pages → **zatech-wiki** → Custom domains
- Click **Set up a custom domain**
- Enter: **wiki.zatech.co.za**
- DNS CNAME record created automatically

2. DNS Configuration (Cloudflare DNS):

Type	Name	Target	Proxy
CNAME	wiki	zatech-wiki.pages.dev	☒ Proxied

3. SSL/TLS:

- Automatic (Universal SSL via Cloudflare)
- Mode: Full (Strict)

2.2 GitHub Actions Workflow

Automated Deployment (`.github/workflows/pages.yml`):

```
name: Deploy to Cloudflare Pages

on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          submodules: recursive # Fetch Hugo themes
          fetch-depth: 0 # Fetch all history for .GitInfo

      - name: Setup Hugo
        uses: peaceiris/actions-hugo@v2
        with:
          hugo-version: '0.150.0'
          extended: true

      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm install

      - name: Build
        run: hugo --minify --gc

      - name: Deploy to Cloudflare Pages
        uses: cloudflare/pages-action@v1
        with:
          apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
          accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
          projectName: zatech-wiki
          directory: public
          githubToken: ${ secrets.GITHUB_TOKEN }
```


Preview Deployments ([.github/workflows/pr-preview.yml](#)):

```
name: Deploy PR Preview

on:
  pull_request:
    branches: [main]

jobs:
  deploy-preview:
    runs-on: ubuntu-latest
    steps:
      # ... same build steps as above ...

  - name: Deploy Preview
    uses: cloudflare/pages-action@v1
    with:
      apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
      accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
      projectName: zatech-wiki
      directory: public
      githubToken: ${ secrets.GITHUB_TOKEN }
      branch: ${ github.head_ref }
```

2.3 Deployment Workflow

Production Deployment

```
# 1. Clone repository (if not already)
git clone https://github.com/zatech/zatech-wiki.git
cd zatech-wiki

# 2. Create branch for changes
git checkout -b content/update-faq

# 3. Make content changes
nano content/faqs/how-to-invite.md

# 4. Test locally
npm install
npm run dev
# Visit: http://localhost:1313

# 5. Commit changes
git add .
git commit -m "Update FAQ: How to invite users"
git push origin content/update-faq

# 6. Create pull request on GitHub
# 7. Review preview deployment
# 8. Merge to main → auto-deploys to production
```

Deployment Timeline:

- Trigger: Push to **main** branch
- Build time: ~60-90 seconds
- CDN propagation: ~30 seconds
- Total time to live: ~2 minutes

Preview Deployments

Trigger: Open pull request

```
# 1. Create feature branch
git checkout -b feature/new-channel-guide

# 2. Add new content
mkdir -p content/channels
nano content/channels/python-channel.md

# 3. Push branch
git push origin feature/new-channel-guide

# 4. Create PR on GitHub

# 5. GitHub Actions automatically:
#   - Builds preview
#   - Posts preview URL in PR comments
#   - Example: https://abc123.zatech-wiki.pages.dev
```

3. Configuration Management

3.1 Hugo Configuration

Main Configuration (`hugo.toml`):

```
# Site information
baseURL = 'https://wiki.zatech.co.za/'
title = 'ZATech Community Wiki'
theme = 'docsy'

# Language
languageCode = 'en-us'
# Content
enableRobotsTXT = true
enableGitInfo = true

# Output formats
[outputs]
home = ['HTML', 'RSS', 'JSON'] # JSON for search index
section = ['HTML', 'RSS']

# Hugo Modules (theme management)
[module]
[[module.imports]]
path = "github.com/google/docsy"
[[module.imports]]
path = "github.com/google/docsy/dependencies"

# Menu configuration
[[menu.main]]
name = 'FAQs'
url = '/faqs/'
weight = 10

[[menu.main]]
name = 'Channels'
url = '/channels/'
weight = 20

[[menu.main]]
name = 'Blog'
url = '/blog/'
weight = 30

# Search configuration
[params.search]
search_index = true
```

3.2 Docsy Theme Configuration

Custom Styles (`assets/scss/_variables_project.scss`):

```
// ZaTech brand colors
$primary: #1a73e8;
$secondary: #34a853;

// Typography
$font-family-sans-serif: -apple-system, BlinkMacSystemFont, "Segoe UI",
Roboto, "Helvetica Neue", Arial, sans-serif;
$font-family-monospace: "SF Mono", Monaco, "Cascadia Code", "Roboto Mono",
monospace;

// Navbar
$navbar-dark-color: #fff;
$navbar-dark-bg: #1a1a1a;
```

Additional Styles (`assets/scss/_styles_project.scss`):

```
// Custom component styles
.hero-section {
  background: linear-gradient(135deg, primary 0%, secondary 100%);
  color: #fff;
}
```

3.3 Content Front Matter

Standard Front Matter (all content files):

```
---
title: "Page Title"
summary: "Short description for previews"
date: 2025-10-30T00:00:00Z
weight: 10 # Sort order (lower = higher priority)
---
```

Blog Post Front Matter:

```
---
title: "Blog Post Title"
date: 2025-10-30T10:00:00Z
summary: "Short summary for listing pages"
author: "Author Name"
tags:
  - community
  - announcement
categories:
  - blog
---
```

3.4 Search Configuration

Enable Search (`hugo.toml`):

```
[params.search]
  search_index = true
  enable_algolia = false # Use Lunr.js (free, client-side)
```

Search Index Output (`config/_default/outputs.toml`):

```
[outputs]
home = ['HTML', 'RSS', 'JSON'] # JSON = search index
```

Hugo generates `/index.json` with searchable content.

4. Content Management

4.1 Writing Guidelines

Markdown Format:

```
---
title: "How to Invite Users to ZaTech Slack"
summary: "Step-by-step guide for inviting new members"
weight: 10
---

## Overview

Brief introduction to the topic.

## Steps

1. First step
2. Second step
3. Third step

## Tips

- Use bullet points for tips
- Keep content concise
- Use code blocks for commands

## See Also

- [Related Guide](/guides/related/)
- [Another Guide](/guides/another/)
```

Best Practices:

- **Headings:** Use `##` for main sections (not `#` - reserved for title)
- **Links:** Use relative links within wiki (`/faqs/topic/`)
- **Images:** Store in `static/images/` directory
- **Code Blocks:** Use triple backticks with language identifier

Example Code Block:

```
# Clone repository
git clone https://github.com/zatech/zatech-wiki.git
```


4.2 Adding New Content

Adding a New FAQ

```
# 1. Navigate to content directory
cd content/faqs

# 2. Create new file
nano how-to-search-slack.md

# 3. Add front matter and content
cat > how-to-search-slack.md << 'EOF'
---
title: "How to Search Slack Messages"
summary: "Tips for effective Slack search"
weight: 15
---

## Using Slack Search

Slack has powerful search capabilities...

EOF

# 4. Preview locally
hugo server

# 5. Commit and push
git add content/faqs/how-to-search-slack.md
git commit -m "Add FAQ: How to search Slack"
git push origin main
```

Adding a Blog Post

```
# 1. Create blog post file
hugo new blog/2025-community-update.md

# 2. Edit file
nano content/blog/2025-community-update.md

# 3. Add content
---
title: "ZaTech Community Update - October 2025"
date: 2025-10-30T10:00:00Z
summary: "Monthly community updates and highlights"
author: "Community Team"
tags:
  - community
  - updates
---

## Introduction

This month's highlights...

# 4. Preview and commit
hugo server
git add content/blog/2025-community-update.md
git commit -m "Add blog: October 2025 update"
git push origin main
```

4.3 Adding Images

```
# 1. Add image to static directory
cp screenshot.png static/images/guides/

# 2. Reference in Markdown
![Screenshot description](/images/guides/screenshot.png)

# 3. With alt text and sizing
![Dashboard view](/images/guides/screenshot.png "Admin dashboard")
```

Image Guidelines:

Image Type	Max Size	Format	Dimensions
Screenshots	200 KB	PNG/WebP	1200x800 max
Diagrams	100 KB	SVG/PNG	800x600 max
Icons	20 KB	SVG	64x64
Photos	150 KB	JPEG/WebP	1000x667 max

4.4 Content Organization

Section Structure:

```

content/
├── _index.md           # Homepage
├── faqs/
│   ├── _index.md      # FAQs section index
│   ├── topic1.md
│   └── topic2.md
├── channels/
│   ├── _index.md      # Channels section index
│   └── channel-name.md
└── blog/
    ├── _index.md      # Blog section index
    └── post-name.md

```

Section Index (`_index.md`):

```

---
title: "FAQs"
menu:
  main:
    weight: 10
---

```

Frequently Asked Questions

Browse our FAQs to find answers to common questions.

4.5 Content Review Process

Pull Request Workflow:

1. **Create PR** with content changes
 2. **Automated Checks:**
 - Hugo build succeeds
 - Markdown linting passes
 - Links are valid
 3. **Manual Review:**
 - Content accuracy
 - Spelling/grammar
 - Formatting consistency
 4. **Preview Deployment:** Review live preview
 5. **Approval:** At least one reviewer approves
 6. **Merge:** Auto-deploys to production
-

5. Build & Release Process

5.1 Local Development

Install Dependencies:

```
# 1. Install Hugo Extended
# macOS:
brew install hugo

# Linux:
wget
https://github.com/gohugoio/hugo/releases/download/v0.150.0/hugo_extended
_0.150.0_linux-amd64.deb
sudo dpkg -i hugo_extended_0.150.0_linux-amd64.deb

# Verify installation
hugo version
# Expected: hugo v0.150.0+extended

# 2. Install Node.js dependencies (PostCSS)
npm install
```

Run Development Server:

```
# Start Hugo dev server
npm run dev
# or
hugo server

# Server runs at: http://localhost:1313
# Auto-reload: Enabled (changes trigger rebuild)
```

Development Server Features:

- **Live Reload:** Automatic browser refresh on file changes
- **Fast Rebuild:** Incremental builds (~50ms)
- **Draft Mode:** Show draft content (`hugo server -D`)

5.2 Production Build

Build Command:

```
# Clean build
rm -rf public resources
npm install
hugo --minify --gc

# Output: public/ directory
```

Build Flags:

Flag	Purpose
<code>--minify</code>	Minify HTML/CSS/JS
<code>--gc</code>	Garbage collection (clean cache)
<code>--baseURL</code>	Override base URL
<code>-D</code>	Include draft content

Build Output:

```
public/
├── index.html           # Homepage
├── faqs/
│   ├── index.html
│   └── topic1/index.html
├── blog/
│   └── post-name/index.html
├── css/
│   └── styles.min.css
├── js/
│   └── main.min.js
├── images/
└── index.json           # Search index
```

6. Monitoring & Performance

6.1 Performance Metrics

Target Metrics (Lighthouse):

Metric	Target	Current
Performance	> 95	98
Accessibility	> 95	100
Best Practices	> 95	100
SEO	> 95	100

Core Web Vitals:

Metric	Target	Typical
LCP (Largest Contentful Paint)	< 2.5s	1.2s
FID (First Input Delay)	< 100ms	50ms
CLS (Cumulative Layout Shift)	< 0.1	0.05

7. Search Functionality

7.1 Search Index Generation

Hugo Search Index (`index.json`):

```
[
  {
    "title": "How to Invite Users",
    "content": "Full page content for indexing...",
    "permalink": "/faqs/how-to-invite/",
    "tags": ["faq", "slack"],
    "summary": "Brief description"
  }
]
```

Configuration (`hugo.toml`):

```
[outputs]
home = ['HTML', 'RSS', 'JSON']

[outputFormats.JSON]
baseName = 'index'
isPlainText = true
mediaType = 'application/json'
```


7.2 Client-Side Search (Lunr.js)

Search Implementation (Docsy theme):

Docsy uses **Lunr.js** for client-side search:

1. Page loads → Download `/index.json` (once)
2. Lunr.js builds search index in browser
3. User searches → Instant results (no server query)

Search Features:

- **Fuzzy Matching:** Typo tolerance
- **Field Boosting:** Title matches ranked higher
- **Stop Words:** Ignore common words (the, and, is)
- **Stemming:** Search "running" finds "run"

7.3 Search Optimization

Improve Search Results:

```
---
title: "Deploy Bot to VPS"
summary: "Step-by-step guide for deploying the ZaTech Slack bot to a VPS"
tags:
  - deployment
  - vps
  - slack
  - docker
---
```

Best Practices:

- Use descriptive titles
- Add relevant tags
- Write clear summaries
- Include synonyms in content (e.g., "VPS" and "Virtual Private Server")

7.4 Search Analytics

Track Search Performance:

```
// Monitor search usage
lunr_search.on('search', (query, results) => {
  console.log(`Search: "${query}" → {results.length} results`);

  // Track low-result searches (content gap?)
  if (results.length < 3) {
    console.warn(`Low results for: "${query}"`);
  }
});
```
